

Full Sharded Data Parallel

Lorem Ipsum

Utshav Saha

sahaushno@gmail.com

11. Sharding: split one big thing into pieces

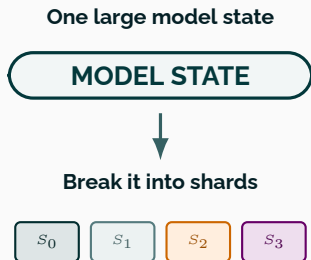
One large model state

MODEL STATE

Story intuition

In games, shards are small pieces that together complete one item. FSDP uses the same idea for training state.

11. Sharding: split one big thing into pieces

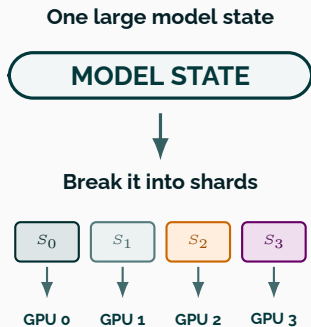


Story intuition

In games, shards are small pieces that together complete one item. FSDP uses the same idea for training state.

- Split one large state into pieces.

11. Sharding: split one big thing into pieces

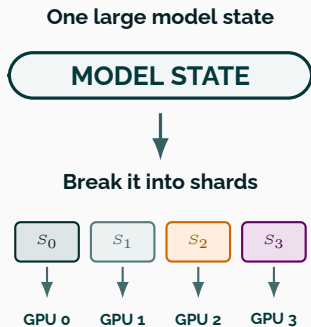


Story intuition

In games, shards are small pieces that together complete one item. FSDP uses the same idea for training state.

- Split one large state into pieces.
- Give different pieces to different GPUs.

11. Sharding: split one big thing into pieces



Story intuition

In games, shards are small pieces that together complete one item. FSDP uses the same idea for training state.

- Split one large state into pieces.
- Give different pieces to different GPUs.
- Reconstruct a full piece of the model only when computation needs it.

FSDP starts by **splitting** model state instead of **replicating** it.

12. What exactly is sharded?

DDP: replicated



Every GPU keeps a full copy.

FSDP: sharded

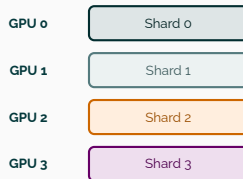
12. What exactly is sharded?

DDP: replicated



Every GPU keeps a full copy.

FSDP: sharded



Each GPU keeps only its fraction.

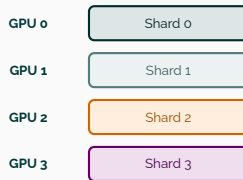
12. What exactly is sharded?

DDP: replicated



Every GPU keeps a full copy.

FSDP: sharded



Each GPU keeps only its fraction.

P = parameters (weights), **G = gradients**, **O = optimizer states** (for Adam, moment estimates). FSDP can shard all three persistent model states.

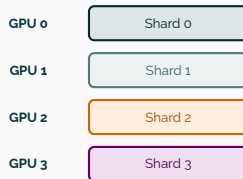
12. What exactly is sharded?

DDP: replicated



Every GPU keeps a full copy.

FSDP: sharded



Each GPU keeps only its fraction.

P = parameters (weights), **G = gradients**, **O = optimizer states** (for Adam, moment estimates). FSDP can shard all three persistent model states.

Idealized persistent model-state memory per GPU scales roughly as $\frac{P+G+O}{N}$.

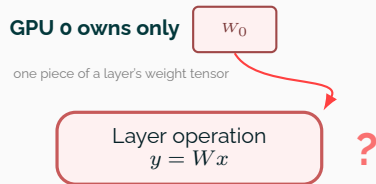
13. The first problem: a layer cannot compute from one shard

GPU 0 owns only

w_0

one piece of a layer's weight tensor

13. The first problem: a layer cannot compute from one shard



The tension

Sharding saves memory, but the current layer normally needs its full parameter tensor to run the matrix operations.

13. The first problem: a layer cannot compute from one shard

GPU 0 owns only

w_0

one piece of a layer's weight tensor

Layer operation
 $y = Wx$

?

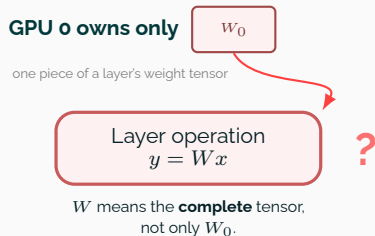
W means the **complete** tensor,
not only w_0 .

The tension

Sharding saves memory, but the current layer normally needs its full parameter tensor to run the matrix operations.

$$w_0 + w_1 + w_2 + w_3$$

13. The first problem: a layer cannot compute from one shard



The tension

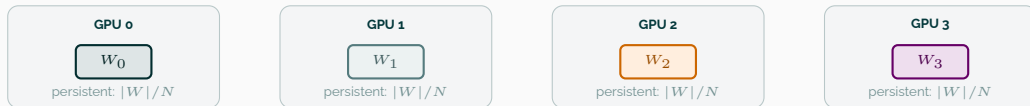
Sharding saves memory, but the current layer normally needs its full parameter tensor to run the matrix operations.



So FSDP must **temporarily collect** the missing parameter shards before computation.

14. All-Gather: temporarily rebuild the current FSDP unit

(a) Before. Each rank persists one disjoint shard W_i of this unit's parameter tensor.



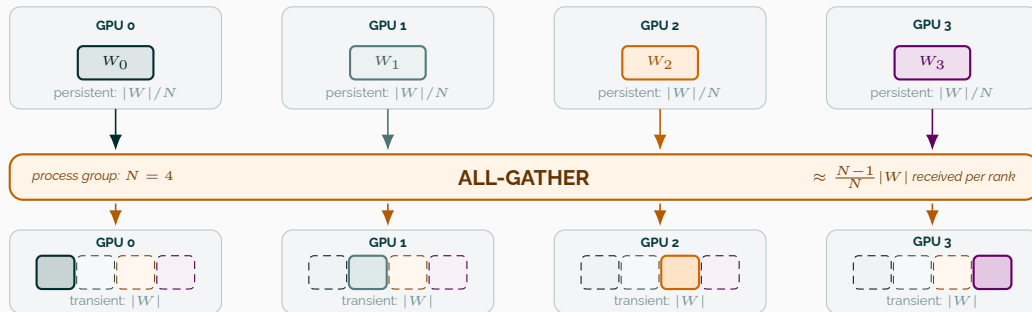
14. All-Gather: temporarily rebuild the current FSDP unit

(a) **Before.** Each rank persists one disjoint shard W_i of this unit's parameter tensor.



14. All-Gather: temporarily rebuild the current FSDP unit

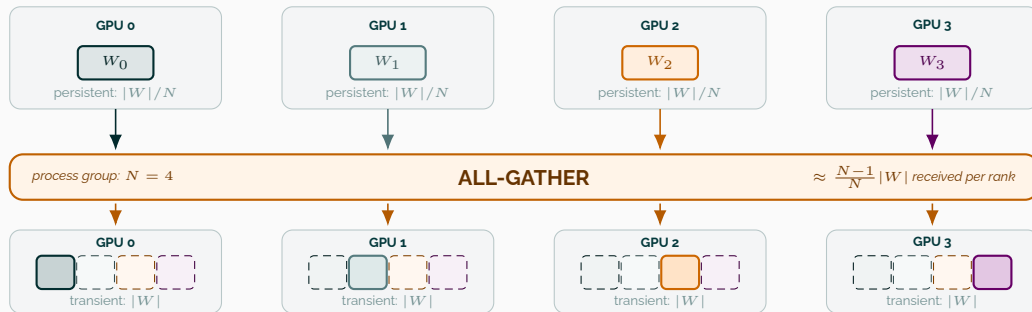
(a) **Before.** Each rank persists one disjoint shard W_i of this unit's parameter tensor.



(b) **After.** Every rank materializes $W = [W_0 \parallel W_1 \parallel W_2 \parallel W_3]$; solid = owned shard, dashed = gathered.

14. All-Gather: temporarily rebuild the current FSDP unit

(a) **Before.** Each rank persists one disjoint shard W_i of this unit's parameter tensor.



(b) **After.** Every rank materializes $W = [W_0 \parallel W_1 \parallel W_2 \parallel W_3]$; solid = owned shard, dashed = gathered.

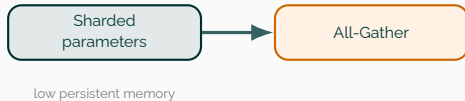
All-Gather: **shards** $\rightarrow$ **full parameters**, but only for the unit currently being computed.

15. Forward pass: gather, compute, reshard

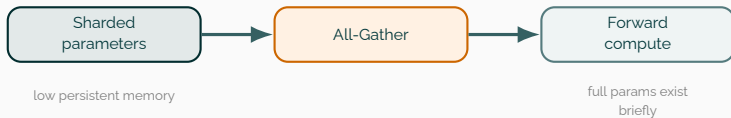
Sharded
parameters

low persistent memory

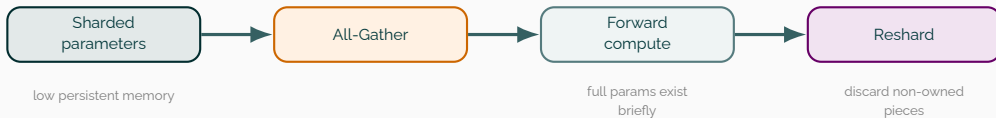
15. Forward pass: gather, compute, reshard



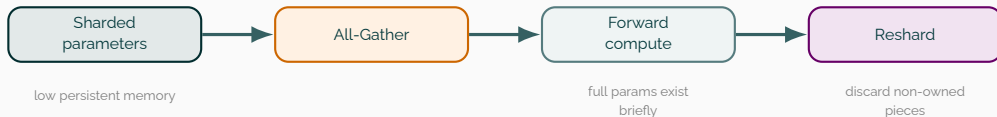
15. Forward pass: gather, compute, reshard



15. Forward pass: gather, compute, reshard



15. Forward pass: gather, compute, reshard



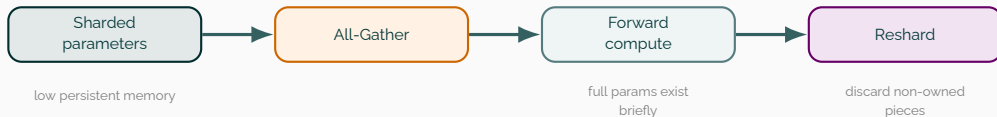
Resharding

After the forward computation, each GPU frees the full parameter and keeps only the shard it owns.

Why it saves memory

The expensive full representation is temporary; the sharded representation is the persistent one.

15. Forward pass: gather, compute, reshard



Resharding

After the forward computation, each GPU frees the full parameter and keeps only the shard it owns.

Why it saves memory

The expensive full representation is temporary; the sharded representation is the persistent one.

Forward = **All-Gather** → **compute** → **reshard**.

16. Simulation: gradients are local at first

After backward on different mini-batches

GPU 0

[2, 1, 0, 3]

What this simulation means

Each GPU trains on a different part of the batch. So after backward, each rank has its own local gradient contribution.

16. Simulation: gradients are local at first

After backward on different mini-batches

GPU 0

[2, 1, 0, 3]

GPU 1

[1, 2, 4, 0]

What this simulation means

Each GPU trains on a different part of the batch. So after backward, each rank has its own local gradient contribution.

16. Simulation: gradients are local at first

After backward on different mini-batches

GPU 0

[2, 1, 0, 3]

GPU 1

[1, 2, 4, 0]

GPU 2

[3, 1, 1, 2]

What this simulation means

Each GPU trains on a different part of the batch. So after backward, each rank has its own local gradient contribution.

16. Simulation: gradients are local at first

After backward on different mini-batches

GPU 0 [2, 1, 0, 3]

GPU 1 [1, 2, 4, 0]

GPU 2 [3, 1, 1, 2]

GPU 3 [0, 2, 3, 1]

What this simulation means

Each GPU trains on a different part of the batch. So after backward, each rank has its own local gradient contribution.

16. Simulation: gradients are local at first

After backward on different mini-batches

GPU 0

[2, 1, 0, 3]

GPU 1

[1, 2, 4, 0]

GPU 2

[3, 1, 1, 2]

GPU 3

[0, 2, 3, 1]

→
These gradients are not synchronized yet.

What this simulation means

Each GPU trains on a different part of the batch. So after backward, each rank has its own local gradient contribution.

Why this is a problem

If each GPU updates its shard using only its local gradient, the model shards will be updated inconsistently.

16. Simulation: gradients are local at first

After backward on different mini-batches

GPU 0 [2, 1, 0, 3]

GPU 1 [1, 2, 4, 0]

GPU 2 [3, 1, 1, 2]

GPU 3 [0, 2, 3, 1]

→
These gradients are not synchronized yet.

What this simulation means

Each GPU trains on a different part of the batch. So after backward, each rank has its own local gradient contribution.

Why this is a problem

If each GPU updates its shard using only its local gradient, the model shards will be updated inconsistently.

Before synchronization: **local gradients differ across GPUs**. We need one agreed global gradient.

17. Simulation: synchronize, then scatter

1. Local gradients

GPU 0 [2, 1, 0, 3]

GPU 1 [1, 2, 4, 0]

GPU 2 [3, 1, 1, 2]

GPU 3 [0, 2, 3, 1]

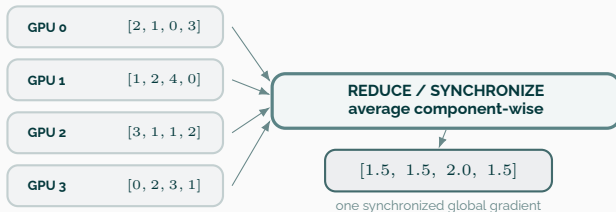
17. Simulation: synchronize, then scatter

1. Local gradients

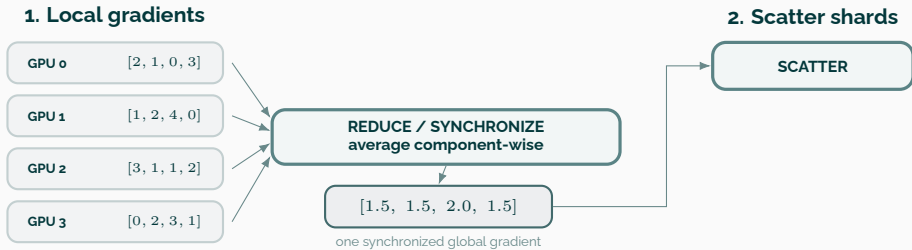


17. Simulation: synchronize, then scatter

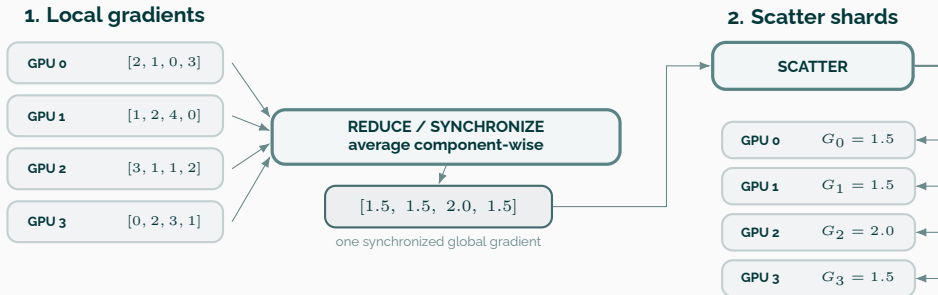
1. Local gradients



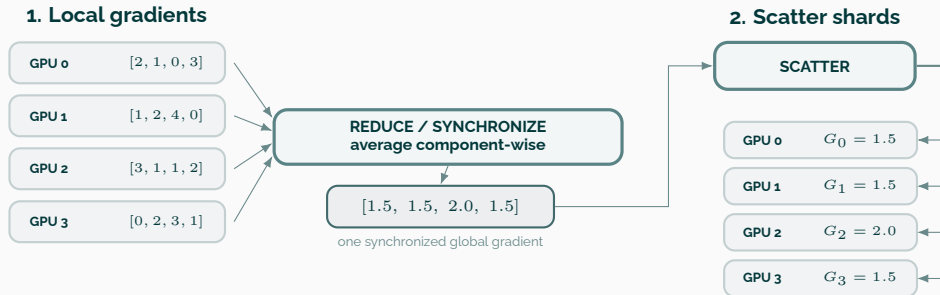
17. Simulation: synchronize, then scatter



17. Simulation: synchronize, then scatter



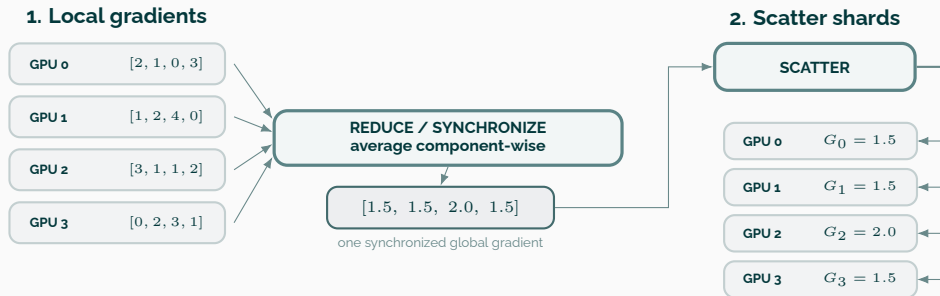
17. Simulation: synchronize, then scatter



What Reduce-Scatter achieves

First, gradients become synchronized by reduction. Then the synchronized result is scattered, so each GPU stores only the gradient shard matching its parameter shard.

17. Simulation: synchronize, then scatter

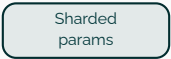


What Reduce-Scatter achieves

First, gradients become synchronized by reduction. Then the synchronized result is scattered, so each GPU stores only the gradient shard matching its parameter shard.

Reduce-Scatter = **synchronize first**, then **keep only the owned gradient shard**.

18. The complete FSDP training cycle



Sharded
params

18. The complete FSDP training cycle



18. The complete FSDP training cycle



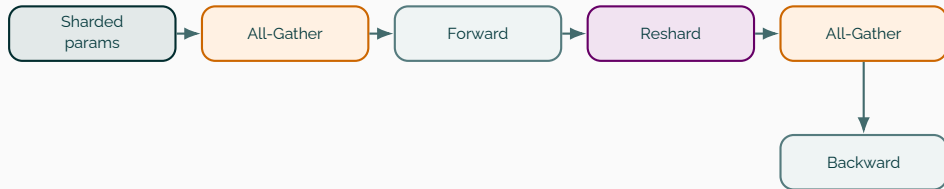
18. The complete FSDP training cycle



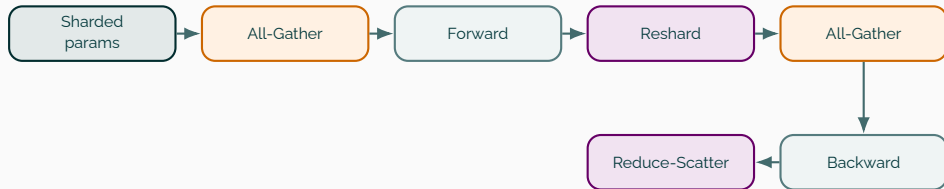
18. The complete FSDP training cycle



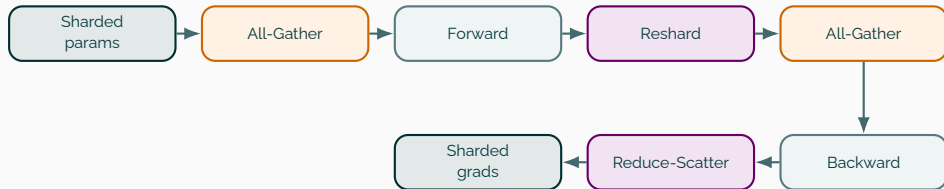
18. The complete FSDP training cycle



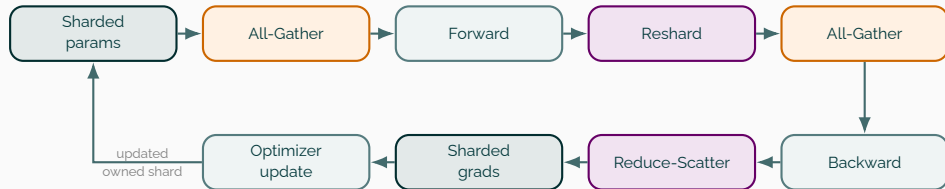
18. The complete FSDP training cycle



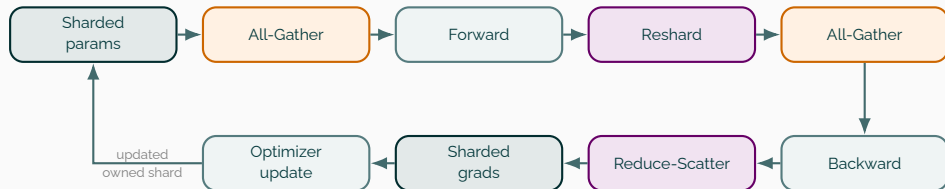
18. The complete FSDP training cycle



18. The complete FSDP training cycle

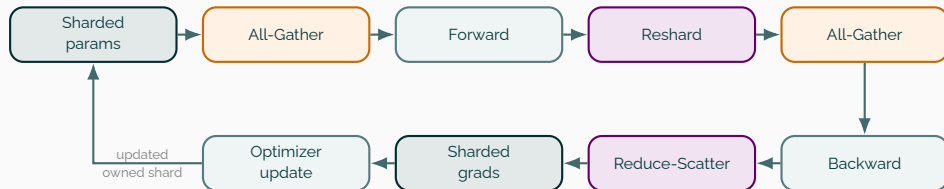


18. The complete FSDP training cycle



At the optimizer step, each GPU already owns the matching **parameter shard + gradient shard + optimizer-state shard**, so it can update that shard locally.

18. The complete FSDP training cycle



At the optimizer step, each GPU already owns the matching **parameter shard + gradient shard + optimizer-state shard**, so it can update that shard locally.

Keep model state divided for memory; gather parameters briefly only when computation needs them.

19. Final trade-off: memory down, communication up

	DDP	FSDP
Model state	Replicated	Sharded
Memory/GPU	Higher	Lower
Parameters	Already local	All-Gather
Gradient sync	All-Reduce	Reduce-Scatter
Main pressure	Memory	Communication

19. Final trade-off: memory down, communication up

	DDP	FSDP
Model state	Replicated	Sharded
Memory/GPU	Higher	Lower
Parameters	Already local	All-Gather
Gradient sync	All-Reduce	Reduce-Scatter
Main pressure	Memory	Communication

The trade

FSDP spends more network communication to reduce persistent GPU memory and make larger models trainable.

19. Final trade-off: memory down, communication up

	DDP	FSDP
Model state	Replicated	Sharded
Memory/GPU	Higher	Lower
Parameters	Already local	All-Gather
Gradient sync	All-Reduce	Reduce-Scatter
Main pressure	Memory	Communication

The trade

FSDP spends more network communication to reduce persistent GPU memory and make larger models trainable.

Overlap hides some cost

Compute block i

Gather block $i + 1$

Communication for the next unit can overlap with current computation.

19. Final trade-off: memory down, communication up

	DDP	FSDP
Model state	Replicated	Sharded
Memory/GPU	Higher	Lower
Parameters	Already local	All-Gather
Gradient sync	All-Reduce	Reduce-Scatter
Main pressure	Memory	Communication

Overlap hides some cost

Compute block i

Gather block $i + 1$

Communication for the next unit can overlap with current computation.

The trade

FSDP spends more network communication to reduce persistent GPU memory and make larger models trainable.

One-sentence summary

Shard most of the time; gather only when needed.

19. Final trade-off: memory down, communication up

	DDP	FSDP
Model state	Replicated	Sharded
Memory/GPU	Higher	Lower
Parameters	Already local	All-Gather
Gradient sync	All-Reduce	Reduce-Scatter
Main pressure	Memory	Communication

Overlap hides some cost

Compute block i

Gather block $i + 1$

Communication for the next unit can overlap with current computation.

The trade

FSDP spends more network communication to reduce persistent GPU memory and make larger models trainable.

One-sentence summary

Shard most of the time; gather only when needed.

FSDP enables larger-model training by trading **memory** for **communication**.